Comp 551 Assignment 3 Group 26

Barry Zhang

March 2026

Abstract

We study the odd-one-out image group classification task from the COMP 551 Mini-Project 3 Kaggle competition, in which each example consists of a group of five grayscale images and the goal is to predict which image is the outlier. This is a structured 5-class classification problem: four images in each group share a latent visual property, while one violates it. We explored two main approaches: a geometric-feature multilayer perceptron (MLP) based on handcrafted shape descriptors, and a Siamese convolutional neural network (CNN) that embeds each image with shared weights and scores its deviation from the group prototype. While our optimized Siamese CNN utilizing CoordConv-style positional channels, dual global pooling, an L_2-based relation module, and an MLP scoring head achieved high internal cross-validation accuracy, it struggled to generalize to the unseen test distribution. Consequently, our final submitted model is the geometric-feature MLP, which captures image spatial statistics and achieved a public Kaggle score of approximately 78.8%. Both architectures satisfy the assignment constraint of at most 25,000 trainable parameters (the MLP at approx. 21,000; the CNN at approx. 22,000). These results illustrate that for small, heavily structured datasets, explicitly encoding geometric inductive bias via handcrafted features can generalize substantially better than learning spatial relationships from raw pixels from scratch.

1 Introduction

The goal of this project is to solve an odd-one-out visual reasoning task in which each input consists of a group of five grayscale images, exactly one of which differs from the other four according to a hidden underlying property. The task is challenging because the relevant property is not given explicitly and can involve subtle geometric or structural regularities rather than simple class membership. Unlike standard image classification, the objective is inherently **relational**: the model must determine which image is inconsistent relative to the rest of its group, rather than recognize each image independently.

Formally, each training example is a set

$$X = \{x_1, x_2, x_3, x_4, x_5\}, \quad x_i \in \mathbb{R}^{1\times 32\times 32}$$

together with a label $y \in \{0, 1, 2, 3, 4\}$, where y denotes the index of the outlier image. The dataset contains **3,000** labeled training groups and **2,000** test groups, with labels for only the first **1,000** test examples provided for public leaderboard evaluation. Since random guessing yields only **20%** accuracy, meaningful performance requires learning nontrivial group-level structure.

We investigated two distinct modeling strategies:

1. **Geometric-Descriptor Relation MLP:** A handcrafted-feature baseline that first extracts contour and geometry-based descriptors from each image, including centroid location, area, perimeter, aspect ratio, convexity, ellipse statistics, symmetry scores, Hu moments, low-frequency radial contour coefficients, and distance-transform thickness features. Each candidate image is then compared to the other four through a relation module that uses the candidate embedding, the mean and standard deviation of the remaining

embeddings, and elementwise difference and interaction terms before an MLP scorer predicts the outlier. This approach provides an interpretable, parameter-efficient model with strong explicit geometric inductive bias.

2. **Siamese CNN:** A deep learning approach that processes each image in the group using a shared convolutional encoder to produce low-dimensional embeddings. These embeddings are then combined through a relation module that compares each image to the group mean embedding, and a small classifier head assigns a score to each candidate outlier. This design allows the model to learn comparative structures directly from raw pixels while remaining within the assignment's strict 25,000 parameter budget.

While our optimized Siamese CNN (utilizing an L_2-based relation representation and an MLP head) achieved high internal cross-validation accuracy, it ultimately suffered from a severe generalization gap on the unseen test set due to its reliance on pixel-level textures rather than true spatial constraints. Consequently, our best overall results were obtained with the handcrafted Geometric MLP, demonstrating the power of explicit geometric inductive bias. The remainder of this report describes our preprocessing procedure, the architecture and training setup of both approaches, the hyperparameter search used to optimize the models, and a comparative analysis of the generalization gap that guided our final model selection.

2 Data Processing

To effectively train our models and prevent overfitting on the relatively small dataset of **3,000** training groups, we implemented a robust data processing pipeline comprising normalization, relation-preserving augmentation, and stratified validation splitting.

Dataset Insights and Preprocessing The dataset consists of groups of five 32×32 grayscale images. Because the objective relies on identifying a structural outlier rather than classifying absolute image content, preserving the relative variations across the five images in a group is paramount. To prepare the data for the PyTorch Siamese CNN, we first expanded the spatial dimensions to include a single channel dimension, resulting in a tensor shape of $(N, 5, 1, 32, 32)$. Furthermore, neural networks are highly sensitive to unscaled input values. We applied global Z-score standardization by calculating the mean and standard deviation across the entire training set, transforming the pixel intensities to have a mean of 0 and a standard deviation of 1. The exact same scaling parameters were subsequently applied to the test set to prevent data leakage and ensure stable gradient flow during inference.

Data Augmentation Data augmentation was necessary to prevent the Siamese CNN from memorizing the specific pixel textures of the training set. Images were augmented using spatial transformation parameters, specifically horizontal flips ($p = 0.5$) and small spatial translations of up to 2 pixels via tensor rolling ($p = 0.7$). This artificially expanded our dataset's spatial variance while strictly preserving the relative geometric properties (such as relative area or center of mass) between the images. Moreover, additive Gaussian noise ($\sigma = 0.03$) was applied to each image to discourage the network from relying on low-level texture artifacts.

Validation Splits To rigorously evaluate our hyperparameter configurations without overfitting to a single static validation set, we utilized a Stratified 3-Fold Cross-Validation approach during the model search phase. Stratification was based on the outlier index label ($y \in \{0, 1, 2, 3, 4\}$) to ensure uniform class distributions across all folds. Once the optimal hyperparameters were identified via the cross-validation search, the final Siamese CNN was trained on the entire available training dataset for a fixed number of epochs.

3 Methods

3.1 Siamese Convolutional Neural Network

Model Architecture To identify the outlier, our Siamese CNN [1] processes all five images through a shared encoder to produce 16-dimensional embeddings (see Figure 1). To inject spatial awareness, we first concatenate normalized X and Y coordinate grids to the input. This 3-channel input passes through two convolutional blocks (32 and 64 filters, utilizing Batch Normalization, ReLU, and Max Pooling). To capture both feature intensity and spatial extent, we concatenate the outputs of Global Max and Average Pooling, projecting the resulting 128-dimensional vector into a 16-dimensional embedding. To determine the outlier, a relation module computes the squared L_2 distance between each individual embedding and the group mean. Finally, a lightweight MLP ($16 \to 16 \to 8 \to 1$) assigns a scalar anomaly score to each image, normalized via a softmax function.

Training Setup The network was trained for 60 epochs (batch size 64) to minimize Cross-Entropy Loss with a 0.05 label smoothing factor to improve generalization. Optimization utilized AdamW ($lr = 1 \times 10^{-3}$, weight decay=1×10^{-4}) alongside gradient clipping (max norm 1.0) to ensure numerical stability. A Cosine Annealing scheduler ($T_{max} = 60$) decayed the learning rate to facilitate smooth convergence. All optimal hyperparameters were identified via 3-Fold Cross-Validation.

Ensuring the Parameter Limit Standard CNNs suffer massive parameter explosions when flattening spatial feature maps into linear layers. By substituting the traditional flattening operation with our dual global pooling strategy, we drastically reduced the input size to the fully connected layers. The base CNN encoder requires $21,648$ parameters and the relational MLP head only 417. In total, our architecture contains $22,065$ trainable parameters, safely satisfying the strict $\leq 25,000$ constraint without sacrificing expressivity.

3.2 Relation-based Descriptor MLP

Model Architecture Rather than classifying raw pixels directly, our final model first converts each 32×32 grayscale tile into a compact geometric descriptor and then performs odd-one-out reasoning over the five descriptors (see Figure 2). Each tile is upscaled to 256×256, smoothed with a Gaussian filter, extracted using Otsu thresholding. From this normalized contour representation, we extract a 59-dimensional feature vector consisting of 53 contour, shape and location. These features encode centroid location, area, perimeter, aspect ratio, convexity, ellipse statistics, radial contour statistics, symmetry scores, Hu moments, low-frequency radial Fourier coefficients, and interior thickness information.

The five tile descriptors are then processed by a shared relation network. First, a shared encoder maps each 59-dimensional tile descriptor to a 32-dimensional embedding using a two-layer MLP: with Layer Normalization, SiLU activations, and dropout after the first hidden layer. Let z_i denote the embedding of tile i. For each candidate tile, we compute the mean embedding of the other four tiles, μ_i, and the elementwise standard deviation of the other four embeddings, σ_i. We then form a relation vector

$$[z_i,\ \mu_i,\ \sigma_i,\ |z_i - \mu_i|,\ z_i \odot \mu_i],$$

which has dimensionality $5 \times 32 = 160$. This relation vector is passed through a lightweight scoring head

$$160 \to 96 \to 1,$$

again with Layer Normalization, SiLU activation, and dropout. The resulting scalar is an anomaly score for that candidate tile. The five anomaly scores are compared with a softmax to produce the final outlier prediction.

Loss Function and Optimization The network is trained using a weighted bootstrap cross-entropy objective combined with a margin-based odd-one-out ranking loss, which improves the robustness of the model. In addition,

we apply an odd-margin term that enforce the differentiation. To further regularize the model, we include both L1 and L2 regulation. Optimization is performed using AdamW. We use label smoothing of 0.06, gradient clipping with maximum norm 0.7, and full permutation training over the five tile positions so that the model remains invariant to slot order. During training, small Gaussian noise is added to the feature vectors to improve robustness.

Hyperparameters and Schedules The final model uses a batch size of 128, learning rate 2×10^{-3}, weight decay 0, manual L_2 regularization $\lambda_{L2} = 3 \times 10^{-5}$, and manual L_1 regularization $\lambda_{L1} = 3 \times 10^{-6}$. The shared encoder width is 64, the embedding size is 32, and the relation head hidden width is 96, with dropout 0.05. Model selection was performed using repeated stratified holdout sweeps rather than a single hand-tuned run. Training uses up to 30 epochs with early stopping patience 6. After selecting the best configuration, the model is retrained on the full training set for 20 epochs. At inference time, we optionally apply descriptor-level test-time augmentation by averaging logits across several very mild preprocessing variants; this does not change the trainable parameter count.

Ensuring the Parameter Limit The model remains under the assignment limit because all heavy geometric processing is performed in fixed preprocessing, not in trainable layers. The neural network only operates on the compact 59-dimensional descriptor for each tile. The trainable parameter count is:

$$(59 \cdot 64 + 64) + 2 \cdot 64$$

for the first linear layer and LayerNorm,

$$(64 \cdot 32 + 32) + 2 \cdot 32$$

for the second linear layer and LayerNorm, and

$$(160 \cdot 96 + 96) + 2 \cdot 96 + (96 \cdot 1 + 1)$$

for the relation head. In total, the architecture contains exactly 21,857 trainable parameters, which satisfies the $\leq$ 25,000 parameter constraint.

4 Results and Analysis

4.1 Results and Analysis: Model Comparison

Internal Validation and Ablation Studies We evaluated several odd-one-out architectures using stratified cross-validation on the 3,000 training groups. The main pattern was that models with stronger relational and geometric inductive bias generalized better than models that attempted to learn similarity directly from raw pixels. In the Siamese CNN family, explicitly constrained comparison rules such as L_1 and L_2 distance performed far better than unconstrained learned similarity mechanisms, indicating that some fixed relational structure is necessary for stable training. More broadly, our experiments showed that descriptor-based relation MLPs were even more reliable, because they operate on contour and geometric features that better match the underlying structure of the task. This suggests that the dataset is governed primarily by global shape relationships rather than local texture or appearance cues.

Table 1: Representative Model Comparison from Internal Validation

Model Family	Relation Type	Main Representation	CV Accuracy
Descriptor MLP	Relation MLP	Geometric / contour features	89%
Siamese CNN	L_2 comparison	Raw pixels	64.63%
Siamese CNN	L_1 comparison	Raw pixels	60.27%
Siamese CNN	Cosine similarity	Raw pixels	27.23%
Siamese CNN	Learned concat	Raw pixels	21.83%

Public Kaggle Score and Generalization Our best Siamese CNN achieved 88.5% internal training accuracy but only **65.70%** on the public Kaggle leaderboard, indicating a substantial generalization gap. By contrast, our descriptor-based relation MLP achieved about 89% internal validation accuracy and 78.8%public accuracy. This gap suggests that raw-pixel CNNs were more prone to learning dataset-specific visual shortcuts, whereas the descriptor-based model was forced to reason through more stable contour and geometric information. Overall, the results indicate that explicitly encoding shape structure provides a much more robust bias for this dataset than learning the full relation end-to-end from pixels.

5 Discussion and Conclusion

Our results show that using explicit image statistics with simple geometric features (center of mass, std. deviation, covariance, etc.) was the main contributor to high performance. The feature-based MLP generalized much better to the test set (78.7% accuracy, not significantly lower than its training performance at 85%-90%) than our other models.

In contrast, the Siamese CNN achieved near-perfect training accuracy but lower test accuracy, implying that it memorized pixel patterns that do not transfer well to "general pattern recognition". This indicates that the task depends strongly on overall spatial structure, rather than on local features. Learning this structure directly from raw pixels is difficult with limited, low-resolution image data.

A limitation of our approach is that the feature extraction step is fixed and only captures the statistics we chose, so it may miss other useful patterns. Effectively, the geometric approach attempts to train the MLP given heuristics that may capture the correct patterns. Another limitation is that the CNN model, while more flexible, did not generalize reliably under the parameter and data constraints of the assignment. In future work, we would test hybrid models that combine learned image embeddings with these engineered features, or experiment with image transforms on the training set, improving the CNN's generalization and reducing overfitting.

6 Statement of Contributions

- **Barry:** Set up the initial project environment; Developed the Relation-based Descriptor MLP model and trained it; co-wrote and edited the report.
- **Maeve:** Developed the Siamese CNN model and implemented simple training; co-wrote and edited the report.
- **Nick:** Performed regularization and hyperparameter tuning on the Siamese CNN model; co-wrote and edited the report.

7 Statement on the Use of LLMs and Other Resources

We used large language model (LLM) tools as writing and development assistants in a limited manner. Specifically, we used ChatGPT and Gemini to improve grammar, clarity, and academic tone, and to suggest LaTeX layout and formatting adjustments. In all cases, we first drafted the report ourselves and then used these tools to refine wording and presentation; the technical content, experimental design, results, and interpretations remain our own. We reviewed and edited all AI-suggested changes to ensure correctness and consistency with our work.

For programming support, LLMs were used for debugging assistance and for generating plotting and visualization code. The core machine learning architectures, specifically the Relation-based Descriptor and the Siamese CNN, were designed, engineered, and trained by our team.

References

[1] R. Nag, "A Comprehensive Guide to Siamese Neural Networks," Medium, 2022. [Online]. Available: https://medium.com/@rinkinag24/a-comprehensive-guide-to-siamese-neural-networks-3358658c0513

A Appendix: Model Architecture Diagrams

This appendix provides the structural visualizations of the two primary models evaluated in this study. Both diagrams outline the flow of data from the input groups to the final relational anomaly scores.

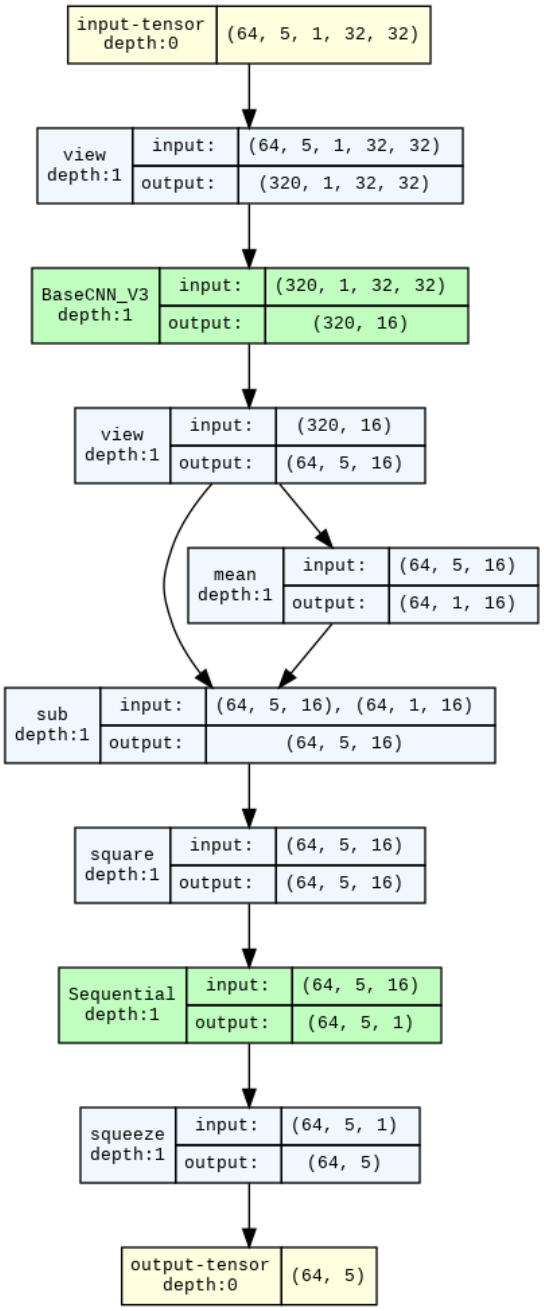


Figure 1: High-level architectural diagram of the Siamese Convolutional Neural Network. The shared convolutional encoder processes the raw 32×32 input images, followed by an L_2 relational distance metric and a Multilayer Perceptron scoring head.

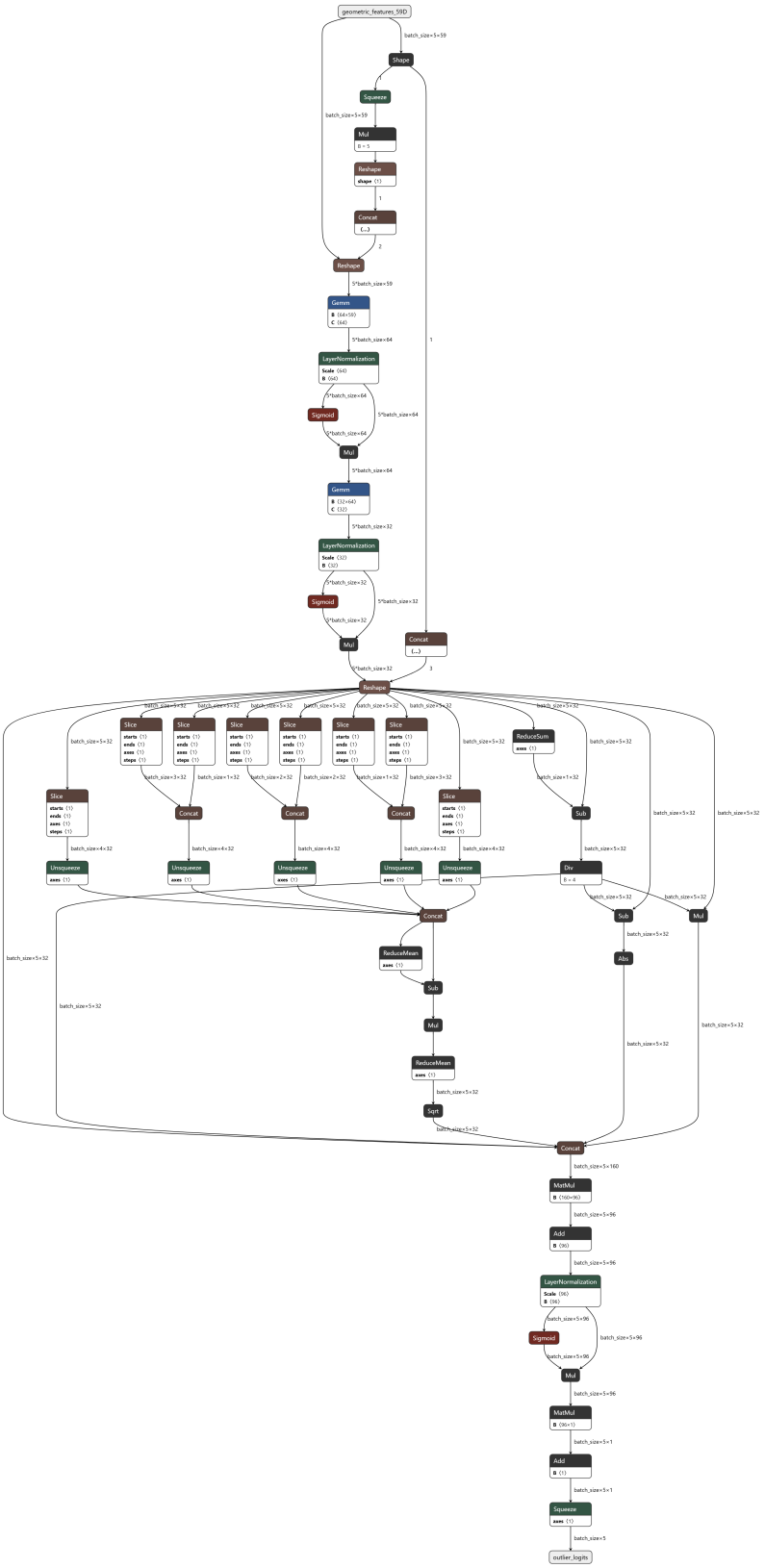


Figure 2: Architectural diagram of the best-performing Geometric-Descriptor Relation MLP. The network processes the 59-dimensional handcrafted geometric feature vectors through sequential SiLU and LayerNorm encoding blocks before computing the group mean/standard deviation and the final outlier scores.